

```

module reg_file (
    input wire clk,
    input wire rst_n, // active low reset
    input wire i_re, // read enable
    input wire i_wr, // write enable
    input wire i_amo_en, // atomic operation enable
    input wire [4:0] i_rs1, // source register 1 address
    input wire [4:0] i_rs2, // source register 2 address
    input wire [4:0] i_rd, // destination register address
    input wire signed [31:0] i_write_data, // data to be written
    input wire [2:0] i_amo_op, // atomic operation selector
    output wire [31:0] o_read_data1, // data read from source register 1
    output wire [31:0] o_read_data2 // data read from source register 2
);

reg [31:0] base_reg[31:0]; // Register file

// Write enable logic
wire is_write;
assign is_write = (i_wr & rst_n & (i_rd != 0));

// Atomic operation result
reg [31:0] amo_result;

always @(*) begin
    case (i_amo_op)
        3'b000: amo_result = base_reg[i_rs1] + i_write_data; // AMOADD
        3'b001: amo_result = base_reg[i_rs1] ^ i_write_data; // AMOXOR
        3'b010: amo_result = base_reg[i_rs1] & i_write_data; // AMOAND
        3'b011: amo_result = base_reg[i_rs1] | i_write_data; // AMOOR
        3'b100: amo_result = i_write_data; // AMOSWAP
        default: amo_result = base_reg[i_rs1];
    endcase
end

// Register initialization
integer i;
always @(posedge clk) begin
    $display("Time=%0t | i_re = %1d | i_wr = %1d | i_amo_en = %1d | i_rs1 = %2d | i_rs2 = %2d | i_write_data = %2d | AMO_OP = %b | basereg [%0d] = %2d | o_read_data1 = %2d | o_read_data2 = %2d",
        $time, i_re, i_wr, i_amo_en, i_rs1, i_rs2, i_write_data, i_amo_op, i_rd,
        base_reg[i_rd-1], o_read_data1, o_read_data2);

    if (~rst_n) begin
        for (i = 0; i < 32; i = i + 1) base_reg[i] <= 32'd0;
    end
end

```

```

        end else if (i_amo_en) begin
            base_reg[i_rd] <= amo_result; // Atomic operation writes result back
        end else if (is_write) begin
            base_reg[i_rd] <= i_write_data;
        end
    end
end

// Asynchronous read
assign o_read_data1 = ((i_rs1 == 0) || ~i_re) ? 32'd0 :
                    (is_write && (i_rs1 == i_rd)) ? i_write_data :
base_reg[i_rs1];
assign o_read_data2 = ((i_rs2 == 0) || ~i_re) ? 32'd0 :
                    (is_write && (i_rs2 == i_rd)) ? i_write_data :
base_reg[i_rs2];

endmodule

```